The slow start implementation begins with a cwd of 1 packet and an initial slow start threshold of 64 packets. The cwd increases by the number of packets acknowledged in each RTT. For congestion avoidance, once cwnd is greater than or equal to ssthresh, then the growth becomes linear. The implementation increases cwnd by packets-acked/cwnd for each ACK received. For fast retransmit, the client tracks duplicate ACKs for each packet. When three duplicate ACKs are received for the same sequence number, the client immediately retransmits the missing packet without waiting for a timeout.

A 500ms timeout is used to detect packet loss. If the ACK is not received during this time, then the client assumes the packet was lost and resends it. For the 1% loss, the cwnd graph has a big growth during the slow start, but dips and evens out a little bit. There are few retransmissions. For the 10% loss, the cwnd graph creates a jagged pattern and the retransmission graphs show a steeper increase in retransmissions as time goes on. For the 50% loss, the cwnd graph shows very uneven and sporadic lines. The retransmissions graph is nearly linear, because the packet loss created the need for more retransmissions.

One of the main challenges I faced was learning how RTT and ssthresh worked with timeouts and retransmissions. Implementing the cwnd update logic gave me some trouble too, because I initially tried updating the cwnd on every ACK, including duplicates, which is incorrect.